# Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud Computing

Data 2026/4/2 Class AI 1

Name Weijie Xu ID 202383910029

## 一、Experimental purpose

(1) To develop a systematic understanding of Docker container behavior, including how containers run based on a single main process and terminate immediately when the process fails or exits. This helps clarify the lifecycle of containers and the importance of correct configuration.

(2) To learn how to expose container services to external users by publishing ports, and understand the mapping relationship between host and container ports, which enables communication between local systems and containerized applications.

(3) To explore how to customize container runtime behavior without modifying the image, including overriding default configurations such as environment variables, resource limits, and startup commands, thereby improving flexibility in deployment.

(4) To understand data management in Docker by comparing ephemeral container storage with persistent storage mechanisms such as volumes, and analyzing how data can be preserved independently of container lifecycles.

(5) To investigate file sharing between the host and containers through bind mounts, and understand how real-time synchronization supports development and debugging workflows.

(6) To build and analyze a multi-container application, including service composition (Nginx, Node.js, Redis), container networking, and inter-service communication using network aliases instead of IP addresses.

(7) To understand reverse proxy and load balancing mechanisms in containerized environments, and analyze how misconfigurations (e.g.incorrect upstream service names) can lead to container startup failures.

(8) To enhance practical debugging skills by analyzing container logs, identifying service dependency issues, and troubleshooting common errors such as failed connections, incorrect network configurations, and misaligned service naming.

## 二、Experimental Contents

- **Container Deployment and Port Publishing**

Containers were created and executed using Docker commands, and port publishing was configured to enable external access. By mapping host ports to container ports, services running inside containers could be accessed through the browser, verifying

the mechanism of communication between the host system and containerized applications.

- **Overriding Container Defaults**

Different runtime configurations were applied to containers without modifying the original images. This included setting environment variables, adjusting port mappings, and customizing startup commands. These operations demonstrated how container behavior can be dynamically controlled at runtime, improving flexibility in deployment and configuration management.

- **Data Persistence with Volumes**

Docker volumes were created and mounted to containers to store data outside the container filesystem. Data was written into the container, and after removing and recreating the container, the stored data remained unchanged. This experiment verified that volumes provide persistent storage independent of container lifecycles.

- **File Sharing with Bind Mounts**

Local directories from the host system were mounted into containers using bind mounts. Files created and modified on the host were immediately reflected inside the container, demonstrating real-time synchronization. This approach is particularly useful for development scenarios where frequent file updates are required.

- **Multi-container Application Setup**

A multi-container system was constructed, including a web application container, a Redis database container, and an Nginx reverse proxy container. A user-defined network was created to enable communication between containers, and service discovery was implemented using network aliases instead of IP addresses.

- **Service Communication and Reverse Proxy Configuration**

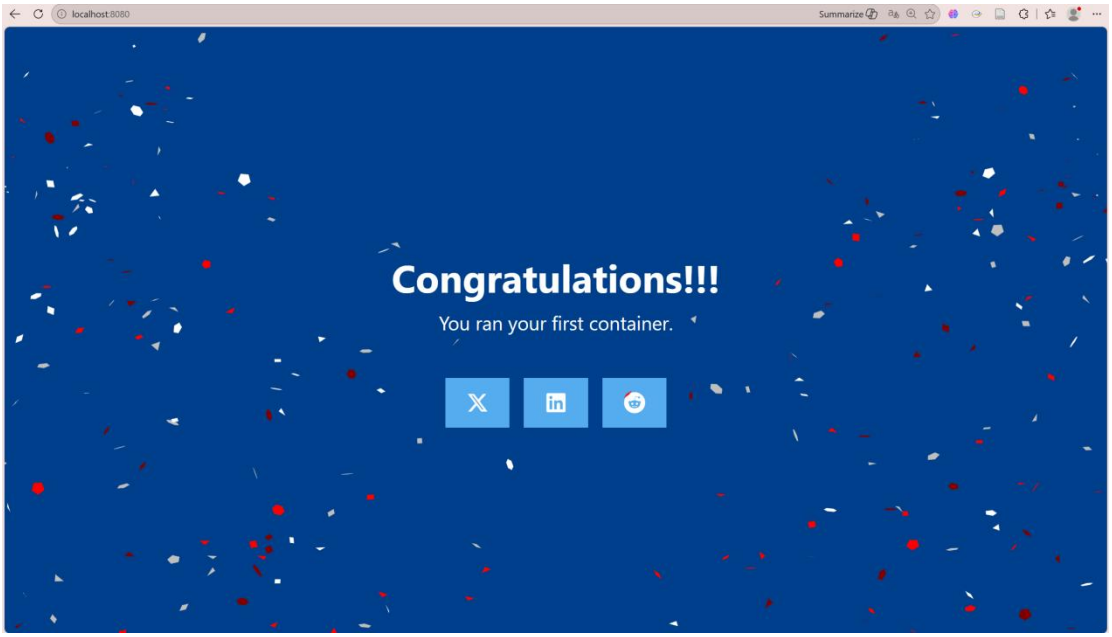
Nginx was configured as a reverse proxy to forward client requests to backend services. Load balancing strategies were explored by defining multiple upstream servers. The experiment also involved troubleshooting configuration errors, such as incorrect service names or ports, which could cause container startup failures.

## 三、Detailed steps of the experiment

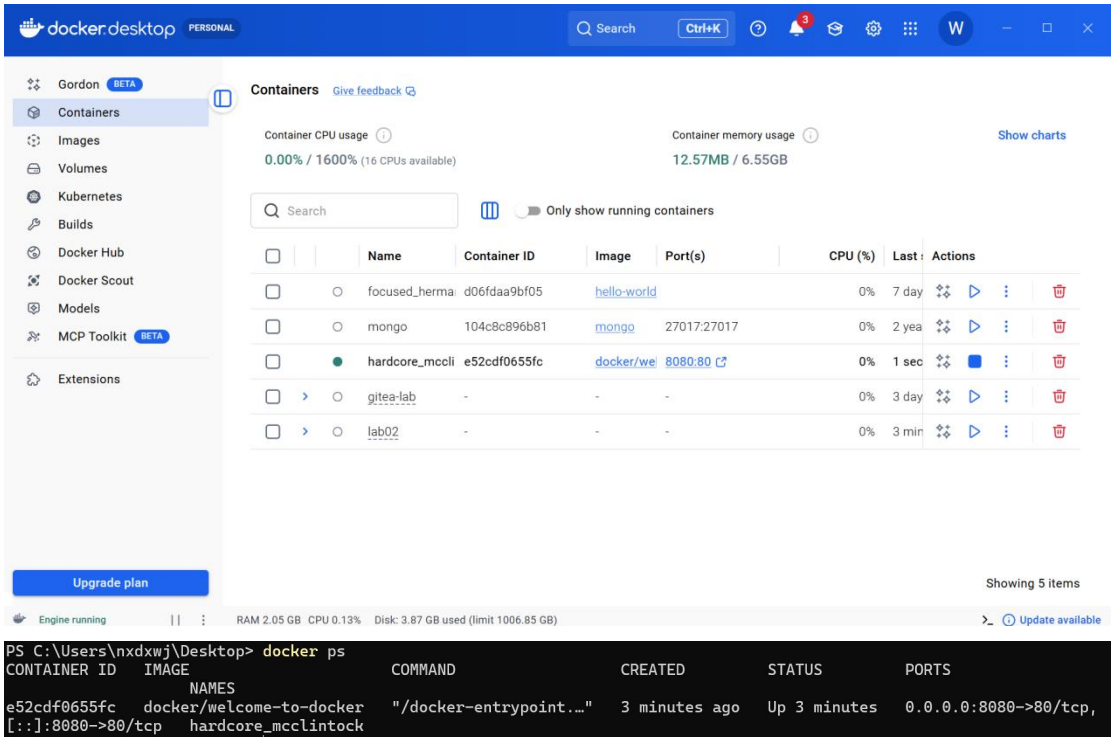
The system downloads the docker/welcome-to-docker image from Docker Hub and starts a container in detached mode, mapping port 8080 on the host to port 80 in the container, enabling access to the application through a web browser.

```
PS C:\Users\nxdxwj\Desktop> docker run -d -p 8080:80 docker/welcome-to-docker
Unable to find image 'docker/welcome-to-docker:latest' locally
latest: Pulling from docker/welcome-to-docker
9824c27679d3: Pull complete
a5585638209e: Pull complete
fd372c3c84a2: Pull complete
958a74d6a238: Pull complete
c1d2dc189e38: Pull complete
828fa206d77b: Pull complete
bdaad27fd04a: Pull complete
9745203f5d34: Pull complete
Digest: sha256:c4d56c24da4f009ecf8352146b43497fe78953edb4c679b841732beb97e588b0
Status: Downloaded newer image for docker/welcome-to-docker:latest
e52cdf0655fc08d64c7792f21018beb05adbeeb5c3b715ccec36d129208ae4d0
```

A Docker container has been successfully deployed and accessed via a local browser. The displayed page confirms that the containerized web application is running correctly and the port mapping is functioning as expected.




Docker Desktop interface showing multiple containers, including a running container with port 8080 mapped to 80, confirming successful deployment and management of containerized applications.



The docker compose up command is starting a containerized service, with logs showing Nginx initialization, configuration loading, and successful startup of worker processes, indicating the application is running normally.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02$ docker compose up
Attaching to app-1
app-1  | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
app-1  | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
app-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
app-1  | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
app-1  | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
app-1  | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
app-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
app-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
app-1  | /docker-entrypoint.sh: Configuration complete; ready for start up
app-1  | 2026/04/02 01:40:44 [notice] 1#1: using the "epoll" event method
app-1  | 2026/04/02 01:40:44 [notice] 1#1: nginx/1.29.0
app-1  | 2026/04/02 01:40:44 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
app-1  | 2026/04/02 01:40:44 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
app-1  | 2026/04/02 01:40:44 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker processes
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 30
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 31
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 32
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 33
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 34
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 35
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 36
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 37
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 38
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 39
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 40
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 41
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 42
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 43
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 44
app-1  | 2026/04/02 01:40:44 [notice] 1#1: start worker process 45
```

The command is pulling the postgres image from Docker Hub and preparing to run a PostgreSQL container, with logs showing the download and extraction of image layers, indicating successful image retrieval.

```
PS C:\Users\nxdxwj\Desktop> docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
ec781dee3f47: Retrying in 1 second
ce2467f2f21d: Retrying in 1 second
d4ced18622af: Retrying in 1 second
af73ff344a10: Waiting
07cec992154a: Waiting
fd0dcffac314: Waiting
bda354b903ce: Waiting
fcd98f4944fb: Waiting
917d5439b698: Waiting
daebffc75218: Waiting
895ed9837058: Waiting
365add159c69: Waiting
f3bba6db66bc: Waiting
latest: Pulling from library/postgres
ec781dee3f47: Pull complete
ce2467f2f21d: Pull complete
d4ced18622af: Pull complete
af73ff344a10: Pull complete
07cec992154a: Pull complete
fd0dcffac314: Pull complete
bda354b903ce: Pull complete
fcd98f4944fb: Pull complete
917d5439b698: Pull complete
daebffc75218: Pull complete
895ed9837058: Pull complete
365add159c69: Pull complete
f3bba6db66bc: Pull complete
Digest: sha256:a9abf4275f9e99bff8e6aed712b3b7dfec9cac1341bba01c1ffdfce9ff9fc34a
Status: Downloaded newer image for postgres:latest
fd89ae9a602050dc47779e261966915d87d60a5464243358d9c5af49c1b4b129
```

A PostgreSQL container is successfully started in detached mode, with environment variables configured and port 5433 on the host mapped to port 5432 in the container.

```
PS C:\Users\nxdxwj\Desktop> docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres
2ddb1743733441dffaf72cabf580b3c3161a18c28341b9cf31d9ddcd8410a264
```

The command runs a PostgreSQL container to display environment variables, showing system configurations and confirming that the custom variable foo=bar has been successfully passed into the container.

```
PS C:\Users\nxdxwj\Desktop> docker run -e foo=bar postgres env
HOSTNAME=6dbcbc4dfc59
PWD=/
HOME=/root
LANG=en_US.utf8
GOSU_VERSION=1.19
foo=bar
PG_MAJOR=18
PG_VERSION=18.3-1.pgdg13+1
SHLVL=0
PGDATA=/var/lib/postgresql/18/docker
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/lib/postgresql/18/bin
```

A PostgreSQL container is launched in detached mode with resource limits applied, including 512 MB memory and 0.5 CPU, demonstrating controlled allocation of system resources.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02$ docker run -d \
-e POSTGRES_PASSWORD=secret \
--memory="512m" \
--cpus="0.5" \
postgres
ea5b6af103cc2ce1cda13774cd3dd0ef9d1984e57ad94dec49add1f1bfe7a5ab
```

The docker stats output displays real-time resource usage of multiple containers, including CPU, memory, network, and I/O metrics, providing insight into container performance and system utilization.

```
CONTAINER ID   NAME                 CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O         PIDS
ea5b6af103cc   quizzical_moser      0.10%     33.03MiB / 512MiB     6.45%     872B / 126B     0B / 40.9MB       9
2ddb17437334   funny_jennings       0.05%     54.29MiB / 6.705GiB   0.79%     1.04kB / 126B   25MB / 41.9MB     9
fd89ae9a6020   cool_matsumoto       0.00%     29.78MiB / 6.705GiB   0.43%     1.17kB / 126B   6.03MB / 42.2MB   9
e52cdf0655fc   hardcore_mcclintock  0.00%     12.65MiB / 6.705GiB   0.18%     1.59kB / 126B   1.82MB / 4.1kB    17

^Z
[1]+  Stopped                 docker stats
```

A custom Docker network is created and listed, followed by launching a PostgreSQL container connected to this network with port mapping configured, enabling inter-container communication and controlled networking.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02$ docker network create mynetwork
8a2dc82343a37a4d7ad3d24db8f5198a58f715d31e1a762936998d4f001e28c1
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02$ docker network ls
NETWORK ID     NAME                      DRIVER    SCOPE
454a997013f4   bridge                    bridge    local
0e505562867a   gitea-lab_gitea           bridge    local
c75803a9a053   gitea-lab_gitea_network   bridge    local
38884093c43e   host                      host      local
a41afcd2c03b   lab02_default             bridge    local
8a2dc82343a3   mynetwork                 bridge    local
fd7f5c44d4eb   none                      null      local
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02$ docker run -d \
-e POSTGRES_PASSWORD=secret \
-p 5434:5432 \
--network mynetwork \
postgres
fa6fcc7a19dc5c348daa0461e2d8bb81026dd098d26fed42737dc48519390261
```

A Docker bridge network is created and verified, then a PostgreSQL container is started and attached to this custom network with port mapping configured, enabling isolated and flexible container networking.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02$ docker run \
  -e POSTGRES_PASSWORD=secret \
  postgres \
  docker-entrypoint.sh -h localhost -p 5432
  The files belonging to this database system will be owned by user "postgres".
  This user must also own the server process.

  The database cluster will be initialized with locale "en_US.utf8".
  The default database encoding has accordingly been set to "UTF8".
  The default text search configuration will be set to "english".

  Data page checksums are enabled.

  fixing permissions on existing directory /var/lib/postgresql/18/docker ... ok
  creating subdirectories ... ok
  selecting dynamic shared memory implementation ... posix
  selecting default "max_connections" ... 100
  selecting default "shared_buffers" ... 128MB
  selecting default time zone ... Etc/UTC
  creating configuration files ... ok
  running bootstrap script ... ok
  performing post-bootstrap initialization ... ok
  syncing data to disk ... ok


  Success. You can now start the database server using:

      pg_ctl -D /var/lib/postgresql/18/docker -l logfile start

  initdb: warning: enabling "trust" authentication for local connections
  initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
  waiting for server to start....2026-04-02 02:28:50.008 UTC [51] LOG:  starting PostgreSQL 18.3 (Debian 18.3-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bi
  t
```

A Docker volume is created and mounted into a container to store data, where a file is written and verified inside the container, demonstrating persistent storage even after the container is removed.

```
PS C:\Users\nxdxwj\Desktop> docker volume create test-volume
test-volume
PS C:\Users\nxdxwj\Desktop> docker run -it --name test-container -v test-volume:/data ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
Status: Downloaded newer image for ubuntu:latest
```

```
root@3d24e273e8de:/# echo "hello docker volume" > /data/test.txt
root@3d24e273e8de:/# cat /data/test.txt
hello docker volume
root@3d24e273e8de:/# exit
docker rm test-container
exit
```

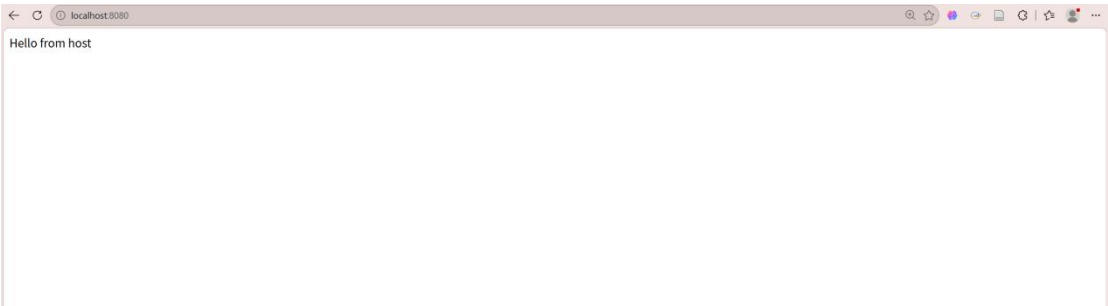
A new container is started with the same Docker volume mounted, and the previously created file is successfully read, confirming data persistence across containers.

```
PS C:\Users\nxdxwj\Desktop> docker run -it --name test-container2 -v test-volume:/data ubuntu
root@7f2c100e8343:/# cat /data/test.txt
hello docker volume
```

A local directory is created and mounted into an Nginx container as a volume, allowing the container to serve a custom HTML file from the host, demonstrating host–container file sharing.

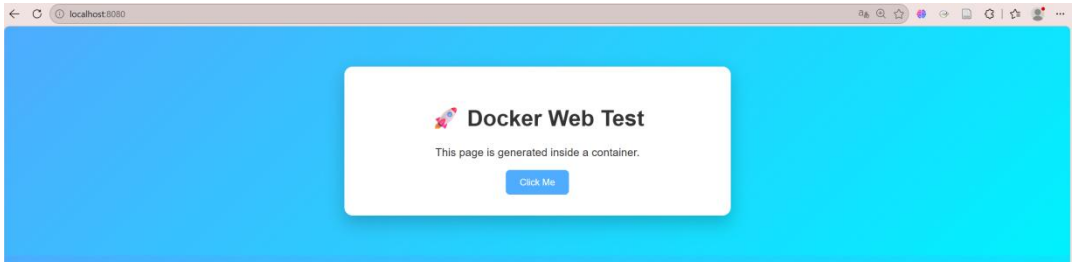
```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02$ mkdir docker-test
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02$ cd docker-test
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02/docker-test$ echo "Hello from host" > index.html
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02/docker-test$ docker run -d \
-p 8080:80 \
-v $(pwd):/usr/share/nginx/html \
nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
ec781dee3f47: Already exists
bb3d0aa29654: Retrying in 2 seconds
510ddf6557d6: Retrying in 2 seconds
cde7a05ae428: Retrying in 2 seconds
587e3d84dbb5: Waiting
3189680c601f: Waiting
5e815e07e569: Waiting
latest: Pulling from library/nginx
ec781dee3f47: Already exists
bb3d0aa29654: Pull complete
510ddf6557d6: Pull complete
cde7a05ae428: Pull complete
587e3d84dbb5: Pull complete
3189680c601f: Pull complete
5e815e07e569: Pull complete
Digest: sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
Status: Downloaded newer image for nginx:latest
e2aa759b002d20d09363944d3d6cf4f548a56e7a8cd6b79530bb8bf53c873bd5_
```

The browser successfully displays content served by the Nginx container, confirming that the host-mounted HTML file is correctly accessed and delivered.




A custom HTML page with CSS styling and interactive elements is created and served through an Nginx container, confirming successful deployment of a dynamic web interface using Docker.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02/docker-test$ cat << 'EOF' > index.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Docker Test Page</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: linear-gradient(135deg, #4facfe, #00f2fe);
            color: #333;
            text-align: center;
            padding: 50px;
        }
        .card {
            background: white;
            border-radius: 12px;
            padding: 30px;
            max-width: 500px;
            margin: auto;
            box-shadow: 0 10px 25px rgba(0,0,0,0.2);
        }
        button {
            padding: 10px 20px;
            border: none;
            background: #4facfe;
            color: white;
            border-radius: 6px;
            cursor: pointer;
        }
        button:hover {
            background: #007bff;
        }
    </style>
</head>
EOFtml>t>ent.getElementById("text").innerText = "🔥 You clicked the button!";
```




A custom Docker network is created, and a Redis container is launched within this network with a network alias assigned, enabling service discovery and communication in a multi-container setup.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02/nginx-node-redis$ docker network create my-app
7fdca5fa18ea8dc16aad9ed011add7f2548dc2221168836593c83fabbbf3c17b
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02/nginx-node-redis$ docker run -d \
--network my-app \
--network-alias redis \
redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
ec781dee3f47: Already exists
5f7274725e4f: Retrying in 1 second
f4f2f7018ed9: Retrying in 1 second
3f63903b0cb8: Retrying in 1 second
c9ff57cee690: Waiting
4f4fb700ef54: Waiting
3e6b2202a764: Waiting
latest: Pulling from library/redis
ec781dee3f47: Already exists
5f7274725e4f: Pull complete
f4f2f7018ed9: Pull complete
3f63903b0cb8: Pull complete
c9ff57cee690: Pull complete
4f4fb700ef54: Pull complete
3e6b2202a764: Pull complete
Digest: sha256:009cc37796fbdbe1b631b4cc0582bed167e5e403ed8bcd06f77eb6cb5aeb6f93
Status: Downloaded newer image for redis:latest
505e2fa1639b8f1178b2979f218a7d395532f5c313dc51c907f017af5eeff13e
```

A custom Docker image is built from a Node.js based Dockerfile, with steps including loading dependencies, installing packages, and packaging application code, resulting in a ready-to-run web service image.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02/nginx-node-redis$ cd web
docker build -t my-web .
[+] Building 60.3s (10/10) FINISHED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
=> [internal] load build definition from Dockerfile                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
=> => transferring dockerfile: 164B                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
=> [internal] load metadata for docker.io/library/node:21                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
=> [internal] load .dockerignore                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
=> => transferring context: 2B                                                                                                                                                                                                                                                                                                                                                                                
=> [1/5] FROM docker.io/library/node:21@sha256:4b232062fa976e3a966c49e9b6279efa56c8d207a67270868f51b3d155c4e33d
=> => resolve docker.io/library/node:21@sha256:4b232062fa976e3a966c49e9b6279efa56c8d207a67270868f51b3d155c4e33d
=> => sha256:4b232062fa976e3a966c49e9b6279efa56c8d207a67270868f51b3d155c4e33d 1.21kB / 1.21kB
=> => sha256:5c8697e822135f8d9b38b54a5aa2bbd3286a092d13a68eb1bc160a883636f9b5 7.41kB / 7.41kB
=> => sha256:c6cf28de8a067787ee0d08f8b01d7f1566a508b56f6e549687b41dfd375f12c7 49.58MB / 49.58MB
=> => sha256:866cb83013581c86f474a7b5530abddcbdefca9559837ee2e0b893ed7e688d39 2.00kB / 2.00kB
=> => sha256:891494355808bdd3db5552f0d3723fd0fa826675f774853796fafa221d850d42 24.05MB / 24.05MB
=> => sha256:6582c62583ef22717db8d306b1d6a0c288089ff607d9c0d2d81c4f8973cbfee3 64.14MB / 64.14MB
=> => sha256:bf2c3e352f3d2eed4eda4feeed44a1022a881058df20ac0584db70c138b041e2 211.21MB / 211.21MB
=> => sha256:2b542796ccab17c22fccf6f4a00414f43ecabd5a5d68fe53f8c8c5aee498b8dd 3.37kB / 3.37kB
=> => sha256:4684276225df21ffeac91884cd316f7bc0d99baeef8ff811f98c2ab46188c3cb 49.72MB / 49.72MB
=> => sha256:08750d53428eebb9ceb8fd6d9516a08420d7747d6a740951c2b043d1ef88c150 1.25MB / 1.25MB
=> => sha256:5d3106ce01c367e8274a6e83a2d6f3d10ecf9619db69a0b3eacebef36f214f88 449B / 449B
=> => extracting sha256:c6cf28de8a067787ee0d08f8b01d7f1566a508b56f6e549687b41dfd375f12c7
=> => extracting sha256:891494355808bdd3db5552f0d3723fd0fa826675f774853796fafa221d850d42
=> => extracting sha256:6582c62583ef22717db8d306b1d6a0c288089ff607d9c0d2d81c4f8973cbfee3
=> => extracting sha256:bf2c3e352f3d2eed4eda4feeed44a1022a881058df20ac0584db70c138b041e2
=> => extracting sha256:2b542796ccab17c22fccf6f4a00414f43ecabd5a5d68fe53f8c8c5aee498b8dd
=> => extracting sha256:4684276225df21ffeac91884cd316f7bc0d99baeef8ff811f98c2ab46188c3cb
=> => extracting sha256:08750d53428eebb9ceb8fd6d9516a08420d7747d6a740951c2b043d1ef88c150
=> => extracting sha256:5d3106ce01c367e8274a6e83a2d6f3d10ecf9619db69a0b3eacebef36f214f88
=> [internal] load build context
=> => transferring context: 16.86kB
=> [2/5] WORKDIR /usr/src/app
=> [3/5] COPY package.json package-lock.json ./
=> [4/5] RUN npm ci
=> [5/5] COPY ./server.js ./
=> exporting to image
=> => exporting layers
=> => writing image sha256:0e97f2f03472e606704b978ad619f1ad75b819211925cdeb19f959aa328f3605
=> => naming to docker.io/library/my-web
```

A custom Nginx Docker image is built using a Dockerfile, including copying a configuration file, resulting in a tailored web server image ready for deployment.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/lab02/nginx-node-redis/web$ cd ../nginx
docker build -t my-nginx .
[+] Building 6.1s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 138B
=> [internal] load metadata for docker.io/library/nginx:1.29
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/nginx:1.29@sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
=> [internal] load build context
=> => transferring context: 210B
=> [2/3] RUN rm /etc/nginx/conf.d/default.conf
=> [3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf
=> exporting to image
=> => exporting layers
=> => writing image sha256:8a928e8f793960c2a6c5bc98aabe15a277f289ed55cad4620587f3590473d000
=> => naming to docker.io/library/my-nginx
```

## 四、Reflection

1. Through this experiment, it became clear that a Docker container is tightly coupled with its main process. If the main process fails to start or exits unexpectedly, the container will terminate immediately. This helped deepen the understanding that many container stops are not normal behavior, but are often caused by configuration errors or service startup failures, especially in cases such as incorrect Nginx configurations.

2. The experiment highlighted how sensitive containerized systems are to configuration consistency. For example, mismatched service names in Nginx upstream settings or incorrect environment variables can cause the entire application to fail. It also demonstrated that multi-container applications rely heavily on correct service dependencies, where failure in one container (e.g. web service) directly affects others (e.g. reverse proxy).

3. By building a multi-container application, the role of Docker networks became

much clearer. Containers communicate through user-defined networks and service names rather than fixed IP addresses. This reinforced the concept of service discovery and showed how network aliases are essential for enabling stable communication between containers.

4. Beyond basic Docker usage, the experiment provided insights into broader system design principles, including microservices architecture, reverse proxy, and load balancing. It demonstrated how complex applications can be decomposed into independent services, each running in its own container and communicating through well-defined interfaces.

# 附录 程序命令代码

附录 1：运行第一个 Web 容器

```
docker run -d -p 8080:80 docker/welcome-to-docker

docker ps

docker compose up
```

附录 2：PostgreSQL 容器运行

```
docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres

docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres

docker run -e foo=bar postgres env
```

附录 3：设置资源限制

```
docker run -d \

    -e POSTGRES_PASSWORD=secret \

    --memory="512m" \

    --cpus="0.5" \

    postgres
```

```
docker stats
```

附录 4：创建 Docker 网络并运行容器

```
docker network create mynetwork

docker network ls

docker run -d \

    -e POSTGRES_PASSWORD=secret \

    -p 5434:5432 \

    --network mynetwork \

    postgres
```

附录 5：Docker Volume 持久化数据

```
docker volume create test-volume

docker run -it --name test-container -v test-volume:/data ubuntu

echo "hello docker volume" > /data/test.txt

cat /data/test.txt

exit

docker rm test-container

docker run -it --name test-container2 -v test-volume:/data ubuntu

cat /data/test.txt
```

附录 6：Bind Mount 挂载本地目录到 Nginx

```
mkdir docker-test

cd docker-test
```

```
echo "Hello from host" > index.html
exit
docker run -d \
    -p 8080:80 \
    -v $(pwd):/usr/share/nginx/html \
    nginx
```

附录 7：配置 Redis 容器

```
docker network create my-app
docker run -d \
    --network my-app \
    --network-alias redis \
    redis
```

附录 8：构建 Node.js Web 镜像

```
cd web
docker build -t my-web .
FROM docker.io/library/node:21
WORKDIR /usr/src/app
COPY package.json package-lock.json ./
RUN npm ci
COPY ./server.js ./
```